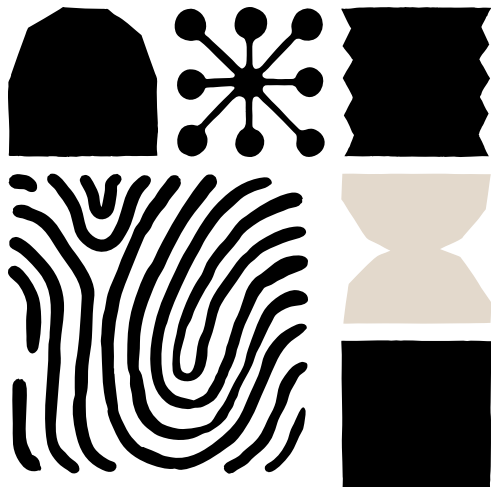


[Research](#)[Economic Futures](#)[Commitments](#) ▾[Learn](#) ▾[News](#)[Try Claude](#)

Engineering at Anthropic



Effective harnesses for long-running agents

Agents still face challenges working across many engineers for inspiration in creating a more effective

Published Nov 26, 2025

As AI agents become more capable, developers take on complex tasks requiring work that spans hours or even days. However,

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our [Cookie Policy](#) [here](#).

[Customize cookie settings](#)[Reject all cookies](#)[Accept all cookies](#)

getting agents to make consistent progress across multiple context windows remains an open problem.

The core challenge of long-running agents is that they must work in discrete sessions, and each new session begins with no memory of what came before. Imagine a software project staffed by engineers working in shifts, where each new engineer arrives with no memory of what happened on the previous shift. Because context windows are limited, and because most complex projects cannot be completed within a single window, agents need a way to bridge the gap between coding sessions.

We developed a two-fold solution to enable the Claude Agent SDK to work effectively across many context windows: an **initializer agent** that sets up the environment on the first run, and a **coding agent** that is tasked with making incremental progress in every session, while leaving clear artifacts for the next session. You can find code examples in the accompanying quickstart.

The long-running agent problem

The Claude Agent SDK is a powerful, general-purpose tool for coding, as well as other tasks that require the ability to read context, plan, and execute. It has context management and compaction, which enables an agent to work on a large context window. Theoretically, given this setup, an agent should be able to continue to do useful work for an arbitrarily long time.

However, compaction isn't sufficient. Out of the box, a model like Opus 4.5 running on the Claude Agent SDK can only maintain a context window of about 128K tokens. This means that even if an agent is able to read and process the entire context window, it can only work on a limited portion of it.

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our [Cookie Policy](#) [here](#).

context windows will fall short of building a production-quality web app if it's only given a high-level prompt, such as “build a clone of claude.ai.”

Claude's failures manifested in two patterns. First, the agent tended to try to do too much at once—essentially to attempt to one-shot the app. Often, this led to the model running out of context in the middle of its implementation, leaving the next session to start with a feature half-implemented and undocumented. The agent would then have to guess at what had happened, and spend substantial time trying to get the basic app working again. This happens even with compaction, which doesn't always pass perfectly clear instructions to the next agent.

A second failure mode would often occur later in a project. After some features had already been built, a later agent instance would look around, see that progress had been made, and declare the job done.

This decomposes the problem into two parts. First, we need to set up an initial environment that lays the foundation for *all* the features that a given prompt requires, which sets up the agent to work step-
Second, we should prompt each agent to make goal while also leaving the environment in a “clean state” we mean the kind of code that we main branch: there are no major bugs, the code and in general, a developer could easily begin having to clean up an unrelated mess.

When experimenting internally, we addressed solution:

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our [Cookie Policy](#) [here](#).

1. **Initializer agent:** The very first agent session uses a specialized prompt that asks the model to set up the initial environment: an `init.sh` script, a `claude-progress.txt` file that keeps a log of what agents have done, and an initial git commit that shows what files were added.
2. **Coding agent:** Every subsequent session asks the model to make incremental progress, then leave structured updates.¹

The key insight here was finding a way for agents to quickly understand the state of work when starting with a fresh context window, which is accomplished with the `claude-progress.txt` file alongside the git history. Inspiration for these practices came from knowing what effective software engineers do every day.

Environment management

In the updated [Claude 4 prompting guide](#), we shared some best practices for multi-context window workflows, including a harness structure that uses “a different prompt for the very first context window.” This “different prompt” requests that the initializer agent set up the environment with all the necessary context that future coding agents will need to

a deeper dive on some of the key components

Feature list

To address the problem of the agent one-shott considering the project complete, we prompt a comprehensive file of feature requirements example prompt. In the [claude.ai](#) clone example, this means a user can open a new chat, type in a query, press

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our [Cookie Policy](#) [here](#).

These features were all initially marked as “failing” so that later coding agents would have a clear outline of what full functionality looked like.

```
{
  "category": "functional",
  "description": "New chat button creates a fresh conversation",
  "steps": [
    "Navigate to main interface",
    "Click the 'New Chat' button",
    "Verify a new conversation is created",
    "Check that chat area shows welcome state",
    "Verify conversation appears in sidebar"
  ],
  "passes": false
}
```

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our [Cookie Policy](#) [here](#).

We prompt coding agents to edit this file only field, and we use strongly-worded instructions or edit tests because this could lead to missing some experimentation, we landed on using JS

likely to inappropriately change or overwrite JSON files compared to Markdown files.

Incremental progress

Given this initial environment scaffolding, the next iteration of the coding agent was then asked to work on only one feature at a time. This incremental approach turned out to be critical to addressing the agent's tendency to do too much at once.

Once working incrementally, it's still essential that the model leaves the environment in a clean state after making a code change. In our experiments, we found that the best way to elicit this behavior was to ask the model to commit its progress to git with descriptive commit messages and to write summaries of its progress in a progress file. This allowed the model to use git to revert bad code changes and recover working states of the code base.

These approaches also increased efficiency, as they eliminated the need for an agent to have to guess at what had happened and get a basic app working again.

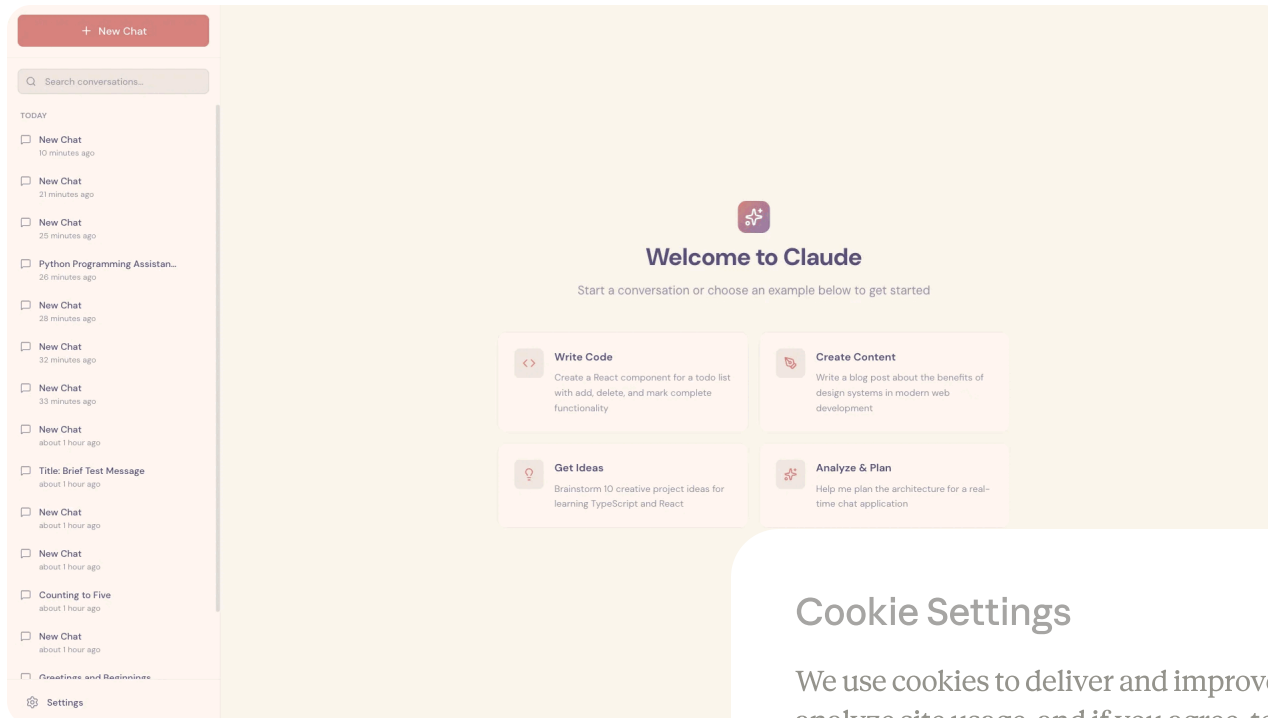
Testing

One final major failure mode that we observed was the agent completing a feature as complete without proper testing. The agent tended to make code changes, and even do test commands against a development server, but the feature didn't work end-to-end.

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our [Cookie Policy](#) [here](#).

In the case of building a web app, Claude mostly did well at verifying features end-to-end once explicitly prompted to use browser automation tools and do all testing as a human user would.



Screenshots taken by Claude through the Puppeteer M clone.

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our [Cookie Policy](#) here.

Providing Claude with these kinds of testing to performance, as the agent was able to identify a...

from the code alone.

Some issues remain, like limitations to Claude's vision and to browser automation tools making it difficult to identify every kind of bug. For example, Claude can't see browser-native alert modals through the Puppeteer MCP, and features relying on these modals tended to be buggier as a result.

Getting up to speed

With all of the above in place, every coding agent is prompted to run through a series of steps to get its bearings, some quite basic but still helpful:

1. Run `pwd` to see the directory you're working in. You'll only be able to edit files in this directory.
2. Read the git logs and progress files to get up to speed on what was recently worked on.
3. Read the features list file and choose the highest-priority feature that's not yet done to work on.

This approach saves Claude some tokens in even figuring out how to test the code. It also helps to run an `init.sh` script that can run the development server, run an end-to-end test before implementing a new feature.

In the case of the claude.ai clone, this meant that the agent could start a local development server and used the Puppeteer MCP to send a message, and receive a response. This ensured that the app was in a good state if the app had been left in a broken state, and it

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our Cookie Policy [here](#).

If the agent had instead started implementing a new feature, it would likely make the problem worse.

Given all this, a typical session starts off with the following assistant messages:

```
[Assistant] I'll start by getting my bearings and understanding the
current state of the project.
[Tool Use] <bash - pwd>
[Tool Use] <read - claude-progress.txt>
[Tool Use] <read - feature_list.json>
[Assistant] Let me check the git log to see recent work.
[Tool Use] <bash - git log --oneline -20>
[Assistant] Now let me check if there's an init.sh script to
restart the servers.
<Starts the development server>
[Assistant] Excellent! Now let me navir
verify that some fundamental features
<Tests basic functionality>
[Assistant] Based on my verification t
fundamental functionality is working w
theme switching, conversation loading,
functioning correctly. Now let me revi
comprehensively to understand what nee
<Starts work on a new feature>
```

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our [Cookie Policy](#) [here](#).

Agent failure modes and solutions

Problem

Initializer Agent Behavior

Coding Agent Behavior

Claude declares victory on the entire project too early.

Set up a feature list file: based on the input spec, set up a structured JSON file with a list of end-to-end feature descriptions.

Read the feature list file at the beginning of a session. Choose a single feature to start working on.

Claude leaves the environment in a state with bugs or undocumented progress.

An initial git repo and progress notes file is written.

Start the session by reading the progress notes file and git commit logs, and run a basic test on the development server to catch any undocumented bugs. End the session by writing a git commit and progress update.

Claude marks features as done prematurely.

Set up a feature list file.

Self-verify all features. Only mark features as “p

Claude has to spend time figuring out how to run

Write an `init.sh` script that can run the deve

Start the session by reading `init.sh`.

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our [Cookie Policy](#) [here](#).

Summarizing four common failure modes and solutions

Future work

This research demonstrates one possible set of solutions in a long-running agent harness to enable the model to make incremental progress across many context windows. However, there remain open questions.

Most notably, it's still unclear whether a single, general-purpose coding agent performs best across contexts, or if better performance can be achieved through a multi-agent architecture. It seems reasonable that specialized agents like a testing agent, a quality assurance agent, or a code cleanup agent, could do an even better job at sub-tasks across the software development lifecycle.

Additionally, this demo is optimized for full-stack web app development. A future direction is to generalize these findings to other fields. It's likely that some or all of these lessons can be applied to the types of long-running agentic tasks required in, for example, scientific research or financial modeling.

Acknowledgements

Written by Justin Young. Special thanks to Dan Jeremy Hadfield, Naia Bouscal, Michael Tingler, Buleandara, Maggie Vo, Pedram Navid, Nadin, and others for their contributions.

This work reflects the collective efforts of several people who made it possible for Claude to safely do long-horizon engineering, especially the code RL & Claude team. People who would like to contribute are welcome to a

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our [Cookie Policy](#) [here](#).

Footnotes

1. We refer to these as separate agents in this context only because they have different initial user prompts. The system prompt, set of tools, and overall agent harness was otherwise identical.

Get the developer newsletter

Product updates, how-tos, community spotlights, and more.
Delivered monthly to your inbox.



Please provide your email address if you'd like to receive our monthly developer newsletter. You can unsubscribe at any time.



Products

Claude

Claude Code

Claude Code Enterprise

Claude Cowork

Claude Code Security

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our [Cookie Policy](#) [here](#).

Claude for Chrome

Claude for Slack

Claude for Excel

Claude for PowerPoint

Claude for Word

Skills

Max plan

Team plan

Enterprise plan

Download app

Pricing

Log in to Claude

Models

Mythos preview

Opus

Sonnet

Haiku

Solutions

AI agents

Code modernization

Coding

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our Cookie Policy [here](#).

Customer support

Education

Financial services

Government

Healthcare

Life sciences

Nonprofits

Security

Claude Platform

Overview

Developer docs

Pricing

Marketplace

Regional compliance

Amazon Bedrock

Google Cloud's Vertex AI

Microsoft Foundry

Console login

Resources

Blog

Claude partner network

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our Cookie Policy [here](#).

Community

Connectors

Courses

Customer stories

Engineering at Anthropic

Events

Inside Claude Code

Inside Claude Cowork

Plugins

Powered by Claude

Service partners

Startups program

Tutorials

Use cases

Help and security

Availability

Status

Support center

Company

Anthropic

Careers

Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our Cookie Policy [here](#).

Economic Futures

Research

News

Claude's Constitution

Responsible Scaling Policy

Security and compliance

Transparency

Terms and policies

Privacy choices

Privacy policy

Consumer health data privacy policy

Responsible disclosure policy

Terms of service: Commercial

Terms of service: Consumer

Usage policy

© 2026 Anthropic PBC



Cookie Settings

We use cookies to deliver and improve our services, analyze site usage, and if you agree, to customize or personalize your experience and market our services to you. You can read our Cookie Policy [here](#).